When AI Agents Work Together

Understanding A2A, MCP, and the Future of Connected AI

Practical demonstrations with Microsoft Agent Framework

Udaiappa Ramachandran (Udai)

CTO|CSO @ Akumina Inc. | Microsoft MVP (AI)

Web: https://udai.io

LinkedIn: https://linkedin.com/in/udair

Meetup: https://meetup.com/nashuaug

Agenda

- The problem, and what A2A is (vs. MCP)
- Core concepts & how the pieces fit together
- How agents talk: transport & interaction patterns
- Maturity: what's stable, what's evolving & what's new
- A2A in the Microsoft Agent Framework
- Why it matters: business value & enterprise use
- Live samples (.NET + Python), patterns & wrap-up

The Problem: Islands of Intelligence



- **Everyone is building agents** — every team, vendor, and framework ships its own
- **Each is a silo** — its own API, memory, tools, and prompts
- **Collaboration means glue code** — custom integration for every pairing
- **No common language** — no shared interoperability standard across these implementations
- **2.5B agents by 2030** — IDC worldwide forecast (Oct 2026)
- **~30,000 agents concurrently** — Anthropic internal R&D platform (Aug 2026)

Sources for the two figures:
https://www.idc.com/resource-center/blog/ai-is-short-of-capacity-not-demand/ (October 22, 2026)
https://www.anthropic.com/institute/measuring-pace-of-ai-development (measurements as of August 2026)

What is A2A?

An open standard for agent-to-agent communication

- **Framework- & vendor-neutral** — agents built by different teams work together
- **Open governance** — created by Google; now hosted by the Agentic AI Foundation (Linux Foundation), alongside MCP
- **Broad backing** — 150+ organizations, incl. AWS, Cisco, Google, IBM, Microsoft, Salesforce, SAP, ServiceNow
- **Protocol milestone** — v1.0 shipped March 2026, with SDKs for Python, JavaScript, Java, C#/.NET, Go, and Rust

A2A vs. MCP — Complementary, Not Competing

MCP — agent to tool

- How an agent connects to APIs, data, and resources
- Equips a single agent with the capabilities it needs
- Think: the agent's hands

A2A — agent to agent

- How independent agents discover and delegate to each other
- Lets specialized agents collaborate across frameworks
- Think: the agent's phone

Use both together. A2A is not an agent framework, a tool-call protocol, or a chat app.

SDK Integration Is Not A2A

Via a vendor SDK

- Bedrock, Gemini, OpenAI, Microsoft Foundry, Copilot Studio
- Called via provider-specific SDKs or REST APIs
- Provider-specific authentication and request models
- May include proprietary discovery or orchestration
- Switching providers often requires integration changes

Via the A2A protocol

- Calls an A2A-compliant endpoint via a standard interface
- Agent Card describes skills, endpoints & security
- Standard messages, tasks, artifacts & operations
- Decouples the caller from the agent's framework
- Compliant agents are easier to add, replace, or compose

It is the protocol interface that makes an interaction A2A — not the model, SDK, framework, or provider. *Some platforms now offer native A2A endpoints or adapters; others still need an A2A gateway or integration layer.*

The Core Actors

User

- Human or automated service
- Defines the goal or request
- Starts the interaction

A2A Client

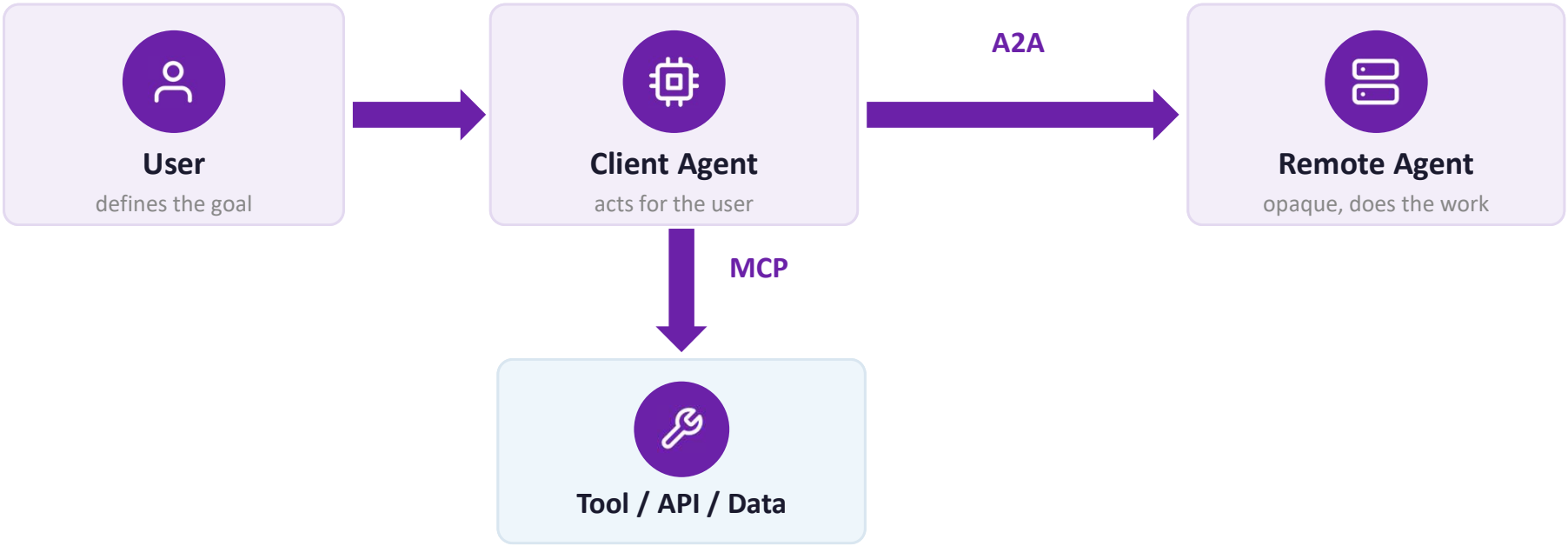
- The client (calling) agent
- Acts on the user's behalf
- Initiates the A2A call

A2A Server

- The remote agent
- Exposes an HTTP endpoint
- Opaque: internals stay private

"Opaque" preserves implementation boundaries — collaborate without exposing internals. Security still comes from auth & policy.

How A2A Works — The Flow



A2A enables direct, standardized collaboration between agents. MCP connects an agent to its tools. The two compose.

The Vocabulary of A2A

- **Agent Card** — JSON "business card": identity, endpoint, skills, auth requirements
- **Task** — a stateful unit of work with an ID and lifecycle (long-running jobs)
- **Message** — a single turn of conversation, role of user or agent
- **Part** — the content inside a message: text, file, or structured data
- **Artifact** — a concrete deliverable produced during a task (doc, image, data)

Agent Cards & Discovery

How agents find and understand each other

- **Published as JSON** — every A2A Server makes an Agent Card available
- **Declares identity & capabilities** — skills, authentication, supported interfaces
- **supportedInterfaces[]** — URL, protocol binding, and protocol version
- **Standard discovery** — /.well-known/agent-card.json
- **Also supported** — registries and direct configuration

```
{
  "name": "Pirate Agent",
  "description": "Speaks like a pirate",
  "version": "1.0.0",
  "supportedInterfaces": [{
    "url": "https://host/a2a/pirate",
    "protocolBinding": "HTTP+JSON",
    "protocolVersion": "1.0"
  }],
  "capabilities": { "streaming": true },
  "securitySchemes": {
    "bearer": { "httpAuthSecurityScheme":
                { "scheme": "Bearer" } }
  },
  "securityRequirements": [
    { "schemes": { "bearer": { "list": [] } } } ],
  "defaultInputModes": ["text/plain"],
  "defaultOutputModes": ["text/plain"],
  "skills": [{ "id": "pirate-chat",
    "name": "Pirate chat",
    "description": "Responds like a pirate",
    "tags": ["fun"] }]
}
```

How Agents Talk

A2A 1.0 defines JSON-RPC, gRPC, and HTTP+JSON bindings (secured via standard transport-level auth)

Result-delivery patterns:

Request / Response

- Send a request, get a reply
- Poll for long-running jobs
- Simplest pattern

Streaming (SSE)

- Open HTTP connection
- Real-time incremental updates
- Great for live UX

Push Notifications

- Server calls your webhook
- For very long / disconnected work
- No open connection needed

Life of a Request: Message vs. Task

Message

- Returned when the agent can answer immediately
- A single turn of communication
- Think: quick Q&A

Task

- Returned for long-running work
- Has an ID and a trackable lifecycle
- Artifacts are returned or updated as work progresses

contextId groups related messages and tasks into one conversation. Message or Task is the server's choice — clients must handle both.

Securing A2A: Auth, Authorization & Tenants

Authentication

The card declares schemes such as OAuth2, OIDC, API key, bearer, or mTLS, using standard HTTP and TLS.

Authorization

The server decides per request and per skill. Return 401/403, and never reveal tasks the caller cannot see.

Tenant isolation

A2A 1.0 carries an optional tenant routing ID on every operation. Isolating data per tenant is your job.

Trusting the card

JWS signatures, checked against trusted keys, verify integrity and publisher; extended cards need sign-in.

A2A standardizes where auth goes. Your identity provider and policies still decide who can do what.

A2A Maturity — Stable Protocol, Rapidly Maturing Ecosystem

Stable A2A 1.0

- Core operations and data model
- Agent Cards & discovery
- Tasks, messages, parts & artifacts
- JSON-RPC, gRPC & HTTP+JSON
- Streaming, polling & push notifications

Still evolving

- SDK/framework package maturity varies
- Provider-native A2A support varies
- Enterprise identity & multi-tenancy patterns
- Production operational tooling
- Interoperability testing across vendors
- Framework-specific adapters

Build against the stable A2A 1.0 protocol; validate your chosen SDK and provider before production.

What's New — Fall 2026

A2A joins the Agentic AI Foundation

Aug 2026: A2A moved into AAIF (Linux Foundation), next to MCP, AGENTS.md, and goose.

MCP spec 2026-07-28

Stateless core, a Tasks extension for long-running work, and enterprise-managed auth.

Official A2A CLI

Oct 2026: discover, message, and stream from any A2A agent in a terminal, test, or CI pipeline.

Agent Framework

Go SDK in public preview (Jul 2026). Core is GA; the A2A packages are still prerelease.

Agent protocols now share one neutral home. MCP gained tasks too, but it is still agent-to-tool.

Where the Microsoft Agent Framework Fits

- **Direct successor** — to Semantic Kernel & AutoGen for agent development and orchestration
- **Multi-language** — .NET and Python (GA since 1.0, Apr 2026), with Go in public preview
- **Expose your agents** — publish them so any A2A client can call
- **Consume remote agents** — call external A2A agents as if they were local
- **First-class A2A** — alongside MCP, AG-UI, and M365 (A2A packages still prerelease)

Why Should I Care?

Lower integration cost

Stop writing custom glue for every agent pairing — connect once, via a standard.

Faster delivery

Compose specialist agents on demand instead of rebuilding capabilities in-house.

Enterprise security alignment

Standard auth patterns; internal prompts, tools & memory stay private. Data disclosure still follows app policy.

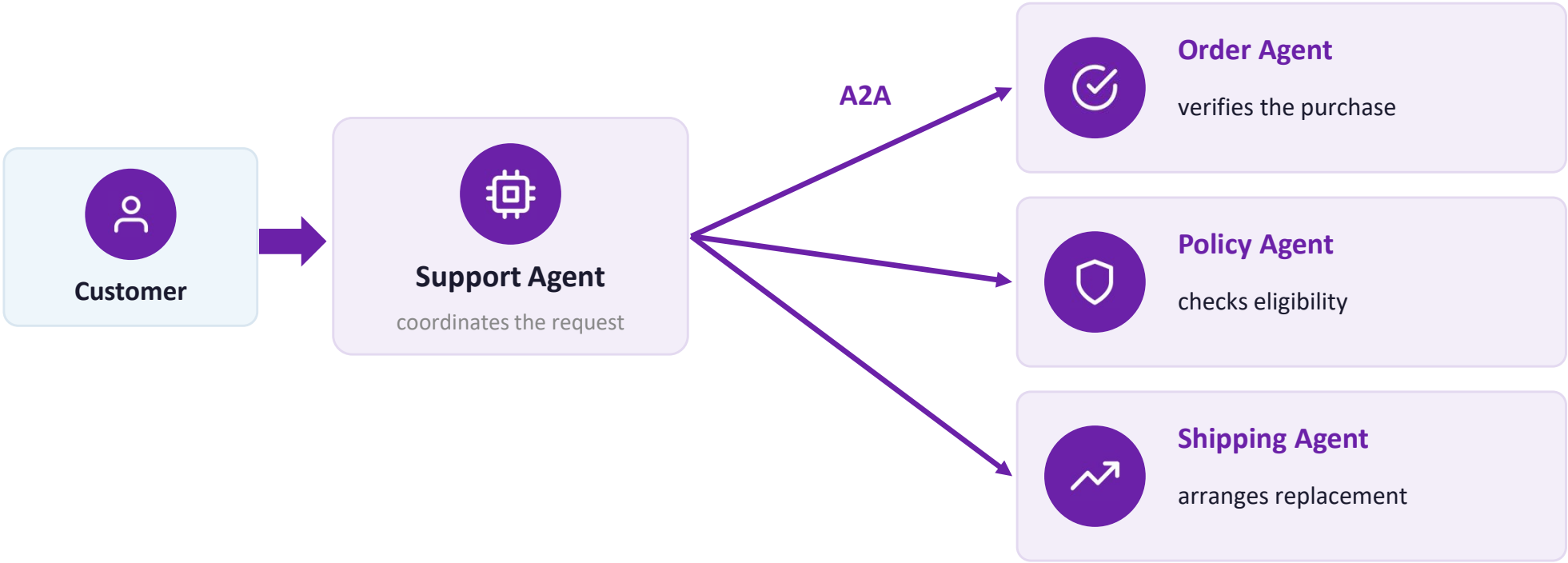
Vendor flexibility

Compose best-of-breed agents across teams and vendors with less integration code and lock-in.

Multi-vendor workflows, specialist teams, partner integrations — without sharing their internal implementation.

One Request, Many Agents

A customer wants to replace a damaged product — the support agent delegates the work over A2A.



Each agent is built and managed independently — they collaborate only through A2A. Swap the shipping agent for a compatible partner agent with minimal changes to the rest of the workflow.

Sample 1 — Expose an Agent over A2A (.NET)

Current Agent Framework hosting pattern (Microsoft.Agents.AI.Hosting.A2A.AspNetCore)

```
var pirateAgent = builder.AddAIAgent(
    "pirate", instructions: "Speak like a pirate.");
pirateAgent.AddA2AServer();
var app = builder.Build();
app.MapA2AHttpJson(pirateAgent, "/a2a/pirate");     // or MapA2AJsonRpc(...)
app.MapWellKnownAgentCard(new AgentCard {
    Name = "Pirate Agent", Description = "Speaks like a pirate", Version = "1.0",
    DefaultInputModes = ["text/plain"], DefaultOutputModes = ["text/plain"],
    Capabilities = new AgentCapabilities { Streaming = true },
    Skills = [ new AgentSkill { Id = "pirate-chat", Name = "Pirate chat",
        Description = "Responds like a pirate", Tags = ["fun"] } ],
    SupportedInterfaces = [ new AgentInterface {
        Url = "http://localhost:5000/a2a/pirate",      // must match the mapped route
        ProtocolBinding = ProtocolBindingNames.HttpJson,
        ProtocolVersion = "1.0" } ]
});
app.Run();
```

MapA2AHttpJson() exposes HTTP+JSON; MapWellKnownAgentCard() serves /.well-known/agent-card.json.
Expected: GET the card → card JSON; POST /a2a/pirate/message:send → "Arrr, ahoy matey!"

Sample 1 — Discover and Call the Agent

A2A 1.0 flow — discover the interface first, then send the message

```
1. Discover
   GET /.well-known/agent-card.json

2. Read supportedInterfaces[]
   HTTP+JSON | JSON-RPC | gRPC

3. Send Message
   role: user  •  parts: [...]  •  contextId: ...

4. Receive
   Message or Task  (optionally streamed as updates)
```

The Agent Card tells the client where and how to communicate with the agent.
Expected: card JSON → client picks HTTP+JSON → reply arrives as a Message (or a Task ID for long work).

Sample 2 — Consume a Remote Agent (Python)

A2AAgent wraps a remote A2A endpoint

```
from agent_framework.a2a import A2AAgent

async with A2AAgent(
    name="remote",
    url="https://agent.example.com"
) as agent:

    response = await agent.run("What are your capabilities?")
    print(response.messages[0].text)
    # → "I can help you search and book flights..."
```

Agent Framework handles discovery and protocol details.
Remote agents can be consumed using a local-agent programming model.

Sample 2 — Streaming & Long-Running Work

Real-time updates and background tasks

```
# Stream tokens in real time (SSE)
stream = agent.run("...", stream=True)
async for update in stream: ...      # → text prints as it arrives

# Long jobs → continuation token to poll later
resp = await agent.run("...", background=True)
if resp.continuation_token:          # None if the server answered directly
    result = await agent.poll_task(resp.continuation_token)
```

Streaming = live, Message-style UX. background=True = Task lifecycle.
Background only works if the remote server returns Tasks — check its support (e.g. .NET-hosted A2A agents don't yet).

Sample 3 — Serve Your Own Agent (Python)

Minimal A2A server — requires installed packages and configured model credentials

```
# pip install agent-framework-a2a "a2a-sdk[http-server]>=1,<2" --pre
import uvicorn
from starlette.applications import Starlette
from a2a.server.request_handlers import DefaultRequestHandler
from a2a.server.routes import create_agent_card_routes, create_jsonrpc_routes
from a2a.server.tasks import InMemoryTaskStore
from a2a.types import AgentCapabilities, AgentCard, AgentInterface
from agent_framework import Agent
from agent_framework.a2a import A2AExecutor
from agent_framework.openai import OpenAIChatClient
agent = Agent(client=OpenAIChatClient(), name="Pirate", instructions="Speak like a pirate.")
card = AgentCard(name="Pirate Agent", description="Speaks like a pirate", version="1.0.0",
    default_input_modes=["text/plain"], default_output_modes=["text/plain"], skills=[],
    capabilities=AgentCapabilities(streaming=True),
    supported_interfaces=[AgentInterface(url="http://localhost:9999/", protocol_binding="JSONRPC")])
handler = DefaultRequestHandler(agent_executor=A2AExecutor(agent, stream=True),
    task_store=InMemoryTaskStore(), agent_card=card)
app = Starlette(routes=[*create_agent_card_routes(card), *create_jsonrpc_routes(handler, "/")])
uvicorn.run(app, host="0.0.0.0", port=9999)
```

In-memory task store and no auth = dev only; use a durable store and your own auth layer in production.
Expected: GET :9999/.well-known/agent-card.json returns the card; any A2A client (e.g. the a2a CLI) can now message it.

Architecture & Patterns

Delegation

- A host agent routes sub-tasks to specialized remote agents

Agents as Tools

- Wrap remote A2A agents as callable tools of a host agent

Cross-Framework Mesh

- Mix Agent Framework, LangGraph, CrewAI, and custom agents

MCP + A2A Together

- Each agent uses MCP for tools, A2A to collaborate

Skip A2A when one team owns both sides in one process — a function call, workflow step, or MCP tool is simpler.

Resources & Q&A

A2A Protocol & Spec

a2a-protocol.org

Agent Framework Repo

github.com/microsoft/agent-framework

A2A SDKs & CLI

github.com/a2aproject (incl. a2a-cli)

Demo Samples

github.com/nhcloud/a2a

Agent Framework A2A Docs

learn.microsoft.com/agent-framework/integrations/a2a

Python A2A Samples

github.com/microsoft/agent-framework (python/samples/04-hosting/a2a)

A2A Samples

github.com/a2aproject/a2a-samples

Start small: expose one agent, or wrap one remote agent, today.

THE TAKEAWAY

A2A lets independent agents collaborate without exposing their internal implementation.

Thank you! Let's connect and build interoperable agents.

Udai — udai.io | linkedin.com/in/udair